VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Komponenta výukového serveru MAV - Kalkulus komunikujících systémů

Component of Learning Server for Modeling and Verification - Calculus of Communicating Systems

Bc. Jakub Koběrský

Diplomová práce

Vedoucí práce: Ing. Martin Kot, Ph.D.

Ostrava, 2026

VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Č.j.VSB/25/140009
vsboee25040DC3

Zadání diplomové práce

Student: **Bc. Jakub Koběrský**

Studijní program: N0613A140034 Informatika

Téma: Komponenta výukového serveru MAV - Kalkulus komunikujících systémů
Component of Learning Server for Modeling and Verification - Calculus of Communicating Systems

Jazyk vypracování: čeština

Zásady pro vypracování:

V rámci diplomových a bakalářských prací vzniká výukový server pro předměty teoretické informatiky. Jedná se o sadu dynamických webových stránek umožňujících studentům pochopení různých typů úloh a problémů tím, že si mohou zadat na stránce libovolné zadání a zobrazí se jim řešení včetně postupu. Cílem této práce je vytvořit komponentu pro výuku vybraných problémů z oblasti formální verifikace systémů.

1. Nastudujte kalkulus komunikujících systémů (CCS), strukturální operační sémantiku (SOS) a ohodnocené přechodové systémy (LTS).
2. Vytvořte dynamické webové stránky umožňující uživateli následující:
a) Zadávat CCS výrazy - program zkontroluje syntaxi, zobrazí syntaktický strom výrazu.
b) Pro 2 CCS výrazy X,Y a daný symbol s tvořit důkaz podle pravidel SOS, zda existuje přechod z X do Y přes s. Tento důkaz bude mít možnost uživatel konstruovat interaktivně (bez nápovědy programu nebo s nápovědou, která pravidla je možné aktuálně použít).
c) Pro zadaný CCS výraz zobrazit odpovídající LTS (nebo jeho část, je-li LTS nekonečný).
d) Mít možnost simulovat běh jednoho CCS i několika paralelně běžících a vzájemně komunikujících CCS.
3. Vytvořte i ukázkové vstupy tak, aby uživatel mohl vše vyzkoušet i bez zadávání vlastních vstupů.

Seznam doporučené odborné literatury:

[1] Luca Aceto, Anna Ingólfsdóttir, Kim G. Larsen and Jiří Srba: Reactive Systems: Modelling, Specification and Verification. Cambridge University Press, August 2007.
[2] Christel Baier, Joost-Pieter Katoen: Principles of Model Checking, The MIT Press, 2008

17. listopadu 2172/15
708 00 Ostrava-Poruba
Česká republika
spojovatelka: +420 597 321 111
epodatelna: epodatelna@vsb.cz
ID datové schránky: d3kj88v
IČ: 61989100
DIČ: CZ61989100
email: studijni.fei@vsb.cz
www.fei.vsb.cz

Formální náležitosti a rozsah diplomové práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí diplomové práce: **Ing. Martin Kot, Ph.D.**

Datum zadání: 01.09.2025

Datum odevzdání: 30.04.2026

Garant studijního programu: prof. RNDr. Václav Snášel, CSc.

V IS EDISON zadáno: 13.10.2025 09:58:20

Abstrakt

TODO

Klíčová slova

diplomová práce; TODO

Abstract

TODO

Keywords

master thesis; TODO

Poděkování

TODO

Obsah

Seznam použitých symbolů a zkratek **8**

Seznam obrázků **9**

Seznam tabulek **10**

1 Úvod **11**

2 Reaktivní systémy **12**

2.1 Rozdíl mezi klasickými a reaktivními systémy 12

2.2 Ohodnocené přechodové systémy 13

2.3 Kalkulus komunikujících systémů 14

2.4 Strukturální operační sémantika 17

2.5 Ekvivalence chování a logické vlastnosti 19

2.6 Rozšíření a další alternativy 20

3 Analýza požadavků **23**

3.1 Účel aplikace 23

3.2 Analýza existujících řešení 23

3.3 Funkční požadavky 24

3.4 Vedlejší požadavky 26

4 Použité technologie **28**

4.1 Vývojové prostředí a sestavení aplikace 28

4.2 Základní webové technologie a uživatelské rozhraní 29

4.3 Zpracování a vizualizace formálních modelů 30

4.4 Správa dat 32

4.5 Doplňkové nástroje 32

5 Návrh řešení **34**
5.1 Případy užití 34
5.2 Návrh datových struktur 35
5.3 Architektura webové aplikace 35
5.4 Uživatelské rozhraní 38

6 Implementace **40**
6.1 Uživatelské rozhraní a logika aplikace 41
6.2 Komunikace mezi komponentami 41
6.3 Možné úpravy aplikace 41
6.4 Kompilace a nasazení 41
6.5 Optimalizace 41
6.6 Možná vylepšení 41

7 Závěr **42**

Literatura **43**

Přílohy **43**

A Struktura přílohy **44**

Seznam použitých zkratek a symbolů

CCS	–	Calculus of Communicating Systems, kalkulus komunikujících systémů
SOS	–	Structural Operational Semantics, strukturální operační sémantika
LTS	–	Labelled Transition System, ohodnocený přechodový systém
AST	–	Abstract Syntax Tree, abstraktní syntaktický strom
HML	–	Hennessy-Milner Logic
LTL	–	Linear Temporal Logic
CTL	–	Computation Tree Logic
CSP	–	Communicating Sequential Processes
ACP	–	Algebra of Communicating Processes
MaV	–	Modelování a verifikace
CAAL	–	Algebra of Communicating Processes
CWB	–	Edinburgh Concurrency Workbench
UX	–	User experience, Uživatelská přívětivost
UI/UX	–	User interface / User experience, Uživatelské rozhraní / Uživatelská přívětivost
PWA	–	Progressive web app, Progresivní webová aplikace
SPA	–	Single Page Application
JS	–	JavaScript
TS	–	TypeScript
CSS	–	Cascading Style Sheets, kaskádové styly
HTML	–	HyperText Markup Language
JSON	–	JavaScript Object Notation
HMR	–	Hot Module Replacement
PEG	–	Parsing Expression Grammar
JSON	–	JavaScript Object Notation

Seznam obrázků

2.1 LTS graf paralelní kompozice $P = a.0 \mid \overline{a}.0$ 16
2.2 LTS graf vynucené synchronizace $Q = (a.0 \mid \overline{a}.0) \setminus \{a\}$ 16

5.1 UML diagram hlavních případů užití aplikace 36
5.2 UML diagram architektury aplikace 37
5.3 Uživatelské rozhraní založené na systému karet 39

Seznam tabulek

Kapitola 1

Úvod

Kapitola 2

Reaktivní systémy

Tradiční pohled na počítačové programy je často zjednodušován na představu černé skříňky, která transformuje zadaný vstup na odpovídající výstup. Takovýto pohled na systém odpovídá popisu algoritmického problému. V praxi se však setkáváme s celou řadou počítačových systémů a aplikací, pro které tento zjednodušený pohled na jednorázovou transformaci není možný [1]. Typickými příklady jsou operační systémy, řídicí jednotky v automobilech, webové servery či běžné bankomaty a nápojové automaty. Tyto systémy, na rozdíl od tradičních algoritmů, nejsou navrženy tak, aby po zpracování dat svůj výpočet ukončily. Jejich primárním účelem je naopak průběžná a nepřetržitá interakce s okolním prostředím.

Tímto prostředím může být uživatel, další softwarové procesy, nebo přímo fyzický svět, ze kterého systém data čte pomocí senzorů. U reaktivních systémů se obecně předpokládá, že mohou teoreticky běžet do nekonečna a že jsou neustále připraveny reagovat na nové události, signály či požadavky pocházející zvenčí. Z tohoto důvodu je nekonečný běh u reaktivních systémů, na rozdíl od klasických programů, zcela přirozený a z hlediska jejich funkce často nezbytný [2].

2.1 Rozdíl mezi klasickými a reaktivními systémy

Základní rozdíl mezi klasickými (transformačními) a reaktivními systémy spočívá v samotné definici jejich korektního chování. U klasických programů, jako jsou například kompilátory, se zaměřujeme primárně na to, aby program pro korektní vstup po konečném počtu kroků skončil a vrátil správný výsledek [3]. Tyto programy lze poměrně dobře modelovat jako matematickou funkci či relaci, která mapuje množinu povolených vstupů na množinu výstupů. Během samotného výpočtu většinou není možné s těmito programy jakkoli interagovat a tím měnit jejich stav. Důkaz korektnosti je zde poměrně jednoduchý, protože často stačí dokázat, že pro každý platný vstup existuje konečný počet kroků, po kterých program skončí a vygeneruje správný výstup [1].

Naproti tomu reaktivní systémy jsou chápány jako nepřetržitě běžící celky. Jelikož u nich nelze snadno definovat konečné stavy značící úspěšný konec výpočtu, nelze je jednoduše zredukovat na

funkci či relaci [1]. Správnost reaktivního systému se nehodnotí jen podle finálního výsledku, ale zejména podle průběhu samotného výpočtu a vzájemné komunikace v čase. Při jejich verifikaci nás proto zajímají, oproti klasickým systémům, zcela jiné vlastnosti. Patří mezi ně bezpečnostní vlastnosti (*safety properties*), které zaručují, že v systému nikdy nenastane nežádoucí stav, jako je například uváznutí (*deadlock*) či porušení vzájemného vyloučení. Dále zkoumáme vlastnosti živosti (*liveness properties*), jež garantují, že se systém nezastaví a dříve či později obslouží každý požadavek [2].

Dalším velkým rozdílem těchto systémů je přítomnost paralelismu, asynchronního chování a s tím spojeného nedeterminismu. Zatímco klasický sekvenční program se pro stejná vstupní data chová vždy deterministicky, reaktivní systémy se typicky skládají z několika nezávislých komponent. Tyto procesy běží souběžně, sdílejí prostředky a reagují na externí události, jejichž přesné načasování nelze předvídat. Právě kvůli tomuto zabudovanému nedeterminismu u nich běžné metody testování často selhávají. Programátor pro ně může vytvořit sadu automatizovaných testů, které pokryjí většinu běžných scénářů, ale to nám nijak nezaručuje, že je systém opravdu bez chyb. Chyby v návrhu, jakými jsou například souběhy (*race conditions*), se často projeví až při velmi specifickém načasování událostí, které se často obtížně odhaluje a replikuje.

Právě tato nepřetržitost, složitá interakce a nedeterminismus jsou hlavními důvody, proč se pro návrh, analýzu a výuku reaktivních systémů používají specializované formální nástroje. Patří mezi ně zejména ohodnocené přechodové systémy (LTS), kterým se detailněji věnuje sekce 2.2, a procesní algebry, kam spadá i kalkulus komunikujících systémů (CCS), popsaný v sekci 2.3, jenž tvoří hlavní náplň praktické části této práce.

2.2 Ohodnocené přechodové systémy

Ohodnocené přechodové systémy (anglicky *Labelled Transition Systems*, zkráceně LTS) představují jeden ze základních formálních modelů pro popis chování reaktivních systémů. Díky této abstrakci můžeme modelovat celý systém jednoduše jako množinu stavů a možných přechodů mezi nimi. Každý z těchto přechodů je navíc ohodnocen určitou akcí, která daný krok popisuje. Tyto akce mohou reprezentovat události, ke kterým v systému dochází, jako je například přijetí vstupu od uživatele, odeslání zprávy po síti, nebo jen interní krok výpočtu. Velkou výhodou tohoto modelu je to, že jej můžeme intuitivně vizualizovat pomocí orientovaných grafů, kde uzly představují jednotlivé stavy systému a hrany značí přechody označené příslušnými akcemi [1].

Výhodou LTS je zejména jeho flexibilita a schopnost zachytit komplexní jevy. Snadno v něm můžeme modelovat jak běžné sekvenční chování, tak i nedeterminismus, kdy z jednoho stavu vede více přechodů se stejnou akcí do různých cílových stavů. Podobně lze pomocí LTS také odhalit stavy uváznutí, ze kterých už nevede žádná další cesta. Díky těmto vlastnostem můžeme přesně definovat a zkoumat vlastnosti systémů, včetně toho, za jakých podmínek považujeme chování dvou systémů za ekvivalentní, například pomocí silné a slabé bisimulace, popsané v sekci 2.5.1.

Množina stavů i přechodů v LTS může být i formálně nekonečná. Jak bude popsáno v následující kapitole 2.3, velkou výhodou procesních algeber je skutečnost, že i konečným zápisem (například za využití rekurze v CCS) dokážeme efektivně definovat a popsat systém s nekonečným stavovým prostorem.

2.2.1 Formální definice LTS

Z matematického hlediska můžeme ohodnocený přechodový systém popsat jako trojici, případně čtveřici, pokud počítáme explicitně i s počátečním stavem [3]. Ohodnocený přechodový systém je definován jako trojice $(Proc, Act, \{\xrightarrow{\alpha} \mid \alpha \in Act\})$, kde:

- $Proc$ je množina stavů (procesů), která může být i nekonečná.
- Act je množina akcí (návěští neboli *labels*).
- $\xrightarrow{\alpha} \subseteq Proc \times Proc$ je binární přechodová relace pro každé $\alpha \in Act$.

Přechod $(s, s') \in \xrightarrow{\alpha}$ lze také chápat jako $(s, \alpha, s') \in \rightarrow$, obvykle se ale značí intuitivnějším zápisem $s \xrightarrow{\alpha} s'$. Tento zápis můžeme číst tak, že systém, který se momentálně nachází ve stavu s, může vykonat akci α a tím přejít do nového stavu s'.

V kontextu modelování pomocí kalkulu komunikujících systémů (CCS) se množina akcí Act rozděluje na dvě hlavní části. První část jsou viditelné akce, které reprezentují přímou komunikaci s okolím (vstupy a výstupy). Druhou částí je speciální neviditelná akce, značená řeckým písmenem τ (tau). Tato neviditelná akce reprezentuje buď interní krok systému, nebo výsledek synchronizace dvou běžících procesů, který zvenčí nelze pozorovat.

2.3 Kalkulus komunikujících systémů

Přestože ohodnocené přechodové systémy poskytují solidní matematický základ pro modelování, jejich přímé využití pro návrh rozsáhlých systémů naráží na limity. V praxi je modelování složitých systémů přímo pomocí LTS velmi zdlouhavé, nepřehledné a náchylné k chybám. Hlavní nevýhodou čistého LTS je nutnost popsat celý zkoumaný systém jako jeden velký a nedělitelný celek. Chybí nám tedy možnost při návrhu poskládat systém z menších nezávislých podsystémů [1].

Tento problém pomáhají řešit procesní algebry, z nichž jednou z nejvýznamnějších je kalkulus komunikujících systémů (anglicky *Calculus of Communicating Systems*, zkráceně CCS). Tento formální jazyk, jehož autorem je držitel Turingovy ceny Robin Milner, si lze představit jako soubor výpočetních agentů komunikujících s okolím prostřednictvím svého definovaného rozhraní [4]. Hlavní výhoda CCS spočívá právě v jeho kompozičnosti. Umožňuje nám definovat chování malých komponent a pomocí sady operátorů je skládat do složitějších celků. V plné šíři své syntaxe (například i při využití operátorů restrikce či přejmenování uvnitř rekurze) je jazyk turingovsky úplný

a dokáže tedy simulovat libovolný Turingův stroj. Jeho účel však nespočívá v provádění klasických algoritmických výpočtů, ale především v modelování a analýze chování interaktivních systémů [3].

2.3.1 Rozhraní a komunikace v CCS

Rozhraní libovolného procesu v CCS si můžeme představit jako sadu komunikačních portů (akcí), přes které proces interaguje se svým okolím, přičemž každý port slouží buď pro vstup, nebo výstup. Komunikace v CCS je založena na principu vzájemné synchronizace dvou procesů (tzv. *handshake* komunikace). Abychom tuto komunikaci mohli modelovat, definuje CCS množinu akcí. Mějme $\mathcal{A}$ jako množinu jmen (reprezentujících například přijetí vstupní hodnoty na portu). Ke každému jménu $a \in \mathcal{A}$ existuje komplementární (opačné) jméno $\overline{a}$ (reprezentující odeslání hodnoty na daném portu). Množinu všech komplementárních jmen označíme jako $\overline{\mathcal{A}}$.

Vedle těchto běžných, navenek viditelných akcí zavádí CCS také již zmíněnou speciální tichou akci τ. Jelikož τ reprezentuje vnitřní krok, o kterém okolí neví, neexistuje k ní žádný komplement. Celková množina všech možných akcí Act v CCS je tedy sjednocením jmen, komplementů a tiché akce: $Act = \mathcal{A} \cup \overline{\mathcal{A}} \cup \{\tau\}$ [1].

2.3.2 Syntaxe CCS

Syntaxe CCS je definována induktivně pomocí relativně malého množství operátorů, které přesto poskytují obrovskou vyjadřovací sílu [1]. Nechť $\alpha \in Act$ je libovolná akce, K je procesní konstanta (jméno procesu) a P je proces. Základní gramatiku jazyka lze poté definovat následovně:

$$P := K \mid \alpha.P \mid P_1 + P_2 \mid P_1|P_2 \mid P \setminus L \mid P[f] \mid 0$$

- **Procesní konstanta ($K \stackrel{\text{def}}{=} P$):** Umožňuje pojmenovávat procesy a odkazovat se na ně, díky čemuž dosahujeme rekurze.

- **Akční prefix ($\alpha.P$):** Proces, který nejdříve vykoná akci α a následně se začne chovat jako proces P, což si můžeme představit jako základní sekvenční operátor.

- **Nedeterministický výběr, suma ($P_1 + P_2$):** Představuje proces, který se může chovat buď jako proces P_1, nebo jako proces P_2. Výběr větve je závislý na tom, která akce je vybrána (ať už interně nebo okolím). Vykonáním akce z jedné větve se druhá alternativa trvale zahodí. Operátor lze také zapsat obecnou sumou $\sum_{i \in I} P_i$, kde I je libovolná množina indexů.

- **Paralelní kompozice ($P_1 \mid P_2$):** Umožňuje procesům P_1 a P_2 běžet souběžně. Procesy mohou vykonávat své akce nezávisle na sobě, nebo se mohou synchronizovat a společně vykonat interní akci τ, pokud je jeden připraven provést akci a a druhý její komplement $\overline{a}$. Tato kompozice je znázorněna na obrázku 2.1.

- **Restrikce ($P \setminus L$):** Kde $L \subseteq \mathcal{A}$. Tento operátor zakazuje (skrývá) vykonání akcí z množiny L a jejich komplementů. Běžně se využívá ve spojení s paralelní kompozicí k vynucení synchronizace mezi dvěma procesy tak, aby akce nebyly dostupné vnějšímu prostředí. Toto chování je znázorněno na obrázku 2.2, kde proces $P' = (a.0 \mid \overline{a}.0) \setminus \{a\}$ může vykonat pouze synchronizaci.

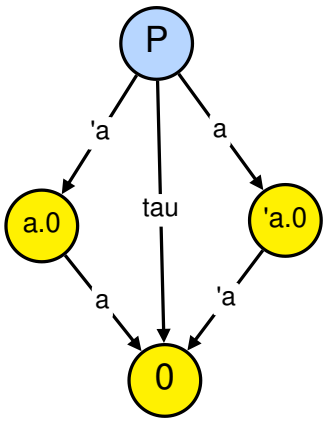
- **Přejmenování ($P[f]$):** Kde $f : Act \to Act$ je funkce s omezeními $f(\tau) = \tau$ a $f(\overline{a}) = \overline{f(a)}$. Tento operátor nám umožňuje znovu použít definované procesy v různých kontextech (např. obecný automat $VM \stackrel{\text{def}}{=} coin.\overline{item}.VM$ můžeme přejmenovat na automat na kávu: $CM \stackrel{\text{def}}{=} VM[coffee/item]$) [1].

- **Prázdný proces (0 nebo *Nil*):** Značí stav, ve kterém proces nemůže vykonat žádnou další akci, a představuje tak korektní ukončení nebo uváznutí (*deadlock*).

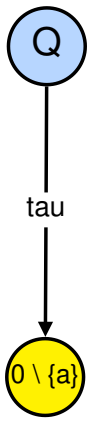
Pro jednoznačnost zápisu se mohu využít závorky. Pokud závorky nejsou přítomny, uplatňuje se následující priorita operátorů (od nejvyšší po nejnižší):

- Restrikce a přejmenování
- Akční prefix
- Paralelní kompozice
- Nedeterministický výběr (suma)

Kompletní model systému pak tvoří *CCS program*, kterým typicky rozumíme konečnou sadu definic procesů.



Obrázek 2.1: LTS graf paralelní kompozice $P = a.0 \mid \overline{a}.0$



Obrázek 2.2: LTS graf vynucené synchronizace $Q = (a.0 \mid \overline{a}.0) \setminus \{a\}$

2.3.3 Vztah mezi CCS a LTS

I když nám syntaxe udává přesná pravidla pro zápis platných výrazů, k pochopení toho, co tyto výrazy skutečně znamenají a jak se chovají v čase, potřebujeme definovat jejich sémantiku. V případě CCS je sémantika formálně určena pomocí strukturální operační sémantiky (SOS), která je popsána v následující kapitole 2.4. V principu ale platí, že každý syntakticky správný CCS výraz můžeme použít jako definici ohodnoceného přechodového systému. Díky tomuto propojení můžeme modely zapsané v CCS analyzovat pomocí algoritmů vytvořených primárně pro LTS a tyto modely graficky vizualizovat (např. s využitím nástroje implementovaného v rámci praktické části této práce). Při návrhu systémů tak můžeme využít modulárního přístupu procesní algebry provázaného s přechodovými systémy.

2.4 Strukturální operační sémantika

Jak již bylo řečeno v sekci 2.3.3, samotná syntaxe jazyka CCS pouze určuje, jak procesy správně zapisovat, neříká nám ale nic o tom, jaký význam tyto zápisy mají a jak se bude popsaný systém reálně chovat. Odvodit odpovídající stavy v modelu LTS je možné i intuitivně, tento postup je však velmi náchylný k chybám a u rozsáhlých systémů jej nelze spolehlivě automatizovat. Právě proto se k tomuto odvození využívá strukturální operační sémantika (SOS), která definuje sadu formálních odvozovacích pravidel [1]. Jejím autorem je Gordon Plotkin a poskytuje exaktní postup, jak z chování jednotlivých malých podkomponent krok za krokem logicky odvodit chování celého komplexního systému.

2.4.1 Odvozovací pravidla

Základním stavebním kamenem strukturální operační sémantiky je systém odvozovacích pravidel (*inference rules*). Tato pravidla nám formálně říkají, za jakých podmínek může náš CCS proces provést určitou akci a přejít do nového stavu. Odvozovací pravidlo se standardně zapisuje ve tvaru [3]:

$$\text{PRAVIDLO} \quad \frac{\text{premisy}}{\text{závěr}} \quad \text{dodatečná podmínka}$$

Tento zápis čteme tak, že pokud platí všechny předpoklady (premisy) nad zlomkovou čarou a je i splněna případná dodatečná podmínka vedle zlomku, pak z toho logicky vyplývá platnost závěru pod čarou. Pravidlům bez premis říkáme axiomy, jejich závěr totiž platí automaticky za všech okolností.

Pro jazyk CCS máme k dispozici ucelenou sadu těchto pravidel pro každý z operátorů. Nechť $\alpha \in Act$ je libovolná akce (viditelná či neviditelná τ) a $a \in \mathcal{A}$ je viditelné jméno akce. Sémantiku operátorů pak můžeme definovat následovně:

- **Akční prefix (ACT):** Toto pravidlo je axiomem a tvoří úplný základ výpočtu, neboť říká, že proces $\alpha.P$ může provést akci α a následně pokračovat jako proces P.

$$\text{ACT} \quad \frac{}{\alpha.P \xrightarrow{\alpha} P}$$

- **Nedeterministický výběr (SUM):** Pravidla definují, že proces poskládaný ze součtu P_1+P_2 může provést buď krok procesu P_1, nebo krok procesu P_2. Provedením vybrané akce druhá větev zcela zaniká. Pravidlo lze také zapsat obecnou sumou.

$$\text{SUM}_1 \quad \frac{P \xrightarrow{\alpha} P'}{P + Q \xrightarrow{\alpha} P'} \qquad \text{SUM}_2 \quad \frac{Q \xrightarrow{\alpha} Q'}{P + Q \xrightarrow{\alpha} Q'}$$

- **Paralelní kompozice (COM):** Paralelní běh modelujeme pomocí tří pravidel. První dvě (COM_1 a COM_2) umožňují takzvané asynchronní prokládání (interleaving), kdy jeden z procesů vykoná svůj krok a druhý zůstává ve svém aktuálním stavu. Třetí pravidlo (COM_3) popisuje synchronizaci, ke které dochází, když jeden proces umí vyslat $\overline{a}$ a druhý přijmout a. Výsledkem této synchronizace je tichá akce τ.

$$\text{COM}_1 \quad \frac{P \xrightarrow{\alpha} P'}{P \mid Q \xrightarrow{\alpha} P' \mid Q} \qquad \text{COM}_2 \quad \frac{Q \xrightarrow{\alpha} Q'}{P \mid Q \xrightarrow{\alpha} P \mid Q'}$$

$$\text{COM}_3 \quad \frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\overline{a}} Q'}{P \mid Q \xrightarrow{\tau} P' \mid Q'}$$

- **Restrikce (RES):** Proces obalený restrikcí se může nadále chovat stejně jako proces původní, avšak s tím rozdílem, že má zakázáno provést jakoukoli akci z množiny zakázaných akcí L, včetně jejího komplementu.

$$\text{RES} \quad \frac{P \xrightarrow{\alpha} P'}{P \setminus L \xrightarrow{\alpha} P' \setminus L} \quad \alpha, \overline{\alpha} \notin L$$

- **Přejmenování (REL):** Pokud vykoná proces nějakou akci α, proces obalený funkcí přejmenování vykoná akci $f(\alpha)$.

$$\text{REL} \quad \frac{P \xrightarrow{\alpha} P'}{P[f] \xrightarrow{f(\alpha)} P'[f]}$$

- **Procesní konstanta (CON):** Toto pravidlo umožňuje nahradit jméno definovaného procesu za jeho vnitřní tělo (proces P) a vykonat krok, který toto tělo nabízí.

$$\text{CON} \quad \frac{P \xrightarrow{\alpha} P'}{K \xrightarrow{\alpha} P'} \quad K \stackrel{\text{def}}{=} P$$

2.4.2 Důkazové stromy (Derivační stromy)

Pomocí výše zmíněných SOS pravidel jsme schopni pro jakýkoli zadaný CCS výraz formálně dokázat, zda je z něj možné provést přechod do jiného stavu přes danou akci [1]. Tento proces dokazování probíhá iterativním aplikováním pravidel, většinou „odspodu nahoru“, čímž postupně budujeme strukturu zvanou odvozovací či důkazový strom (*derivation tree*). Důkazový strom je také možné sestrojit opačným směrem, kdy začínáme axiomem a aplikujeme pravidla do té doby, než dosáhneme požadovaného výrazu.

Kořenem takového stromu je tvrzení o přechodu, jehož existenci chceme ověřit, a jednotlivá zanoření a větvení představují pravidla, která jsme v daném kroku použili. Aby byl důkaz považován za platný, musí listy stromu tvořit výhradně axiomy (většinou se jedná o pravidlo ACT), u kterých již není třeba nic dalšího dokazovat. Pokud se nám pro testovaný přechod podaří zkonstruovat validní konečný strom zakončený platnými axiomy, včetně splnění všech dodatečných podmínek u zlomků, tak dostaneme matematický důkaz, že přechod skutečně existuje.

Příkladem derivačního stromu může být proces s rekurzivní definicí $A \stackrel{\text{def}}{=} a.A$. Pro přechod $((A \mid \overline{a}.0) \mid b.0)[c/a] \xrightarrow{c} ((A \mid \overline{a}.0) \mid b.0)[c/a]$ by příslušný strom vypadal následovně [3]:

$$\text{REL}\ \dfrac{\text{COM}_1\ \dfrac{\text{COM}_1\ \dfrac{\text{CON}\ \dfrac{\text{ACT}\ \dfrac{}{a.A \xrightarrow{a} A}}{A \xrightarrow{a} A}\ A \stackrel{\text{def}}{=} a.A}{A \mid \overline{a}.0 \xrightarrow{a} A \mid \overline{a}.0}}{(A \mid \overline{a}.0) \mid b.0 \xrightarrow{a} (A \mid \overline{a}.0) \mid b.0}}{((A \mid \overline{a}.0) \mid b.0)[c/a] \xrightarrow{c} ((A \mid \overline{a}.0) \mid b.0)[c/a]}$$

2.5 Ekvivalence chování a logické vlastnosti

Při navrhování systémů a tvoření modelů velmi často narazíme na potřebu porovnat dva nezávislé procesy a dokázat, že dělají „to samé“. Zatímco u běžných sekvenčních programů bychom pouze srovnali výstupní data pro stejné vstupy, v prostředí neustále interagujících procesů toto neplatí. Nástrojem pro posuzování takové shody u reaktivních systémů je zkoumání takzvané ekvivalence chování a dokazování vlastností pomocí modálních logik [1].

2.5.1 Ekvivalence chování

Klíčovým konceptem pro určení ekvivalence v LTS a CCS je bisimulace. Její hlavní myšlenka spočívá v tom, že stavy systému (či procesy) považujeme za ekvivalentní ve chvíli, kdy dokážou navzájem napodobovat své přechody do nekonečna. Základní a nejpřísnější formou je silná bisimulace (*strong bisimulation*). Největší takovou relaci pak nazýváme silná bisimilarita, značená operátorem $\sim$. Dva procesy jsou silně bisimulární (tedy existuje mezi nimi silná bisimulace) právě tehdy, když na každý krok prvního procesu umí druhý proces odpovědět ekvivalentním identickým krokem pod stejnou

akcí, a to včetně skrytých tichých akcí τ. Jelikož jde o symetrickou relaci, musí toto pravidlo platit obousměrně i pro stavy, do kterých se oba procesy přesunou. Silnou bisimulaci lze formálně dokázat například přes hru útočníka a obránce. Pro reálné systémy je tento způsob často příliš striktní.

Pro nasazení v reálných aplikacích se proto spíše používá slabá bisimulace (*weak bisimulation*) a z ní plynoucí slabá bisimilarita, značená symbolem $\approx$. Při ní nás zajímá pouze viditelné chování (vstupy a výstupy), zatímco interní výpočty můžeme ignorovat [1]. Slabá bisimulace systémům dává větší volnost a umožňuje jim před nebo po provedení viditelné akce vykonat libovolný počet neviditelných τ přechodů. Tímto přístupem lépe odráží perspektivu vnějšího pozorovatele, protože abstrahuje od vnitřního fungování komponent.

2.5.2 Hennessy-Milner logika

Kromě ověřování ekvivalence dvou celých systémů (*Equivalence Checking*) využíváme ve formální verifikaci také techniku ověřování modelů (*Model Checking*). Při ní testujeme, jestli konkrétní systém splňuje námi požadovanou vlastnost. Touto vlastností může být například podmínka, že se automat po stisknutí tlačítka dříve nebo později vrátí do výchozího stavu. Existuje mnoho nástrojů pro specifikaci těchto vlastností, příkladem mohou být logiky Linear Temporal Logic (LTL), Computation Tree Logic (CTL) nebo expresivní modální μ-kalkul. Mezi jedny z hlavních nástrojů pro formulaci těchto lokálních vlastností patří také právě Hennessy-Milner logika (HML).

Jedná se o modální logiku, která rozšiřuje klasickou výrokovou logiku o dva speciální operátory, kterými jsou možnost ($\langle a \rangle F$) a nutnost ($[a]F$) [3]:

- Formule $\langle a \rangle F$ je splněna ve stavu s, pokud z tohoto stavu existuje *alespoň jeden* přechod přes akci a do stavu, který splňuje formuli F.

- Formule $[a]F$ je splněna ve stavu s, pokud *všechny* možné přechody přes akci a vedou do stavů, které splňují formuli F. Tuto formuli triviálně splňuje i jakýkoliv stav, ze kterého pod akcí a žádný přechod vůbec nevede.

O dvou procesech, které mají konečné větvení (*image-finite* LTS), platí tvrzení, že jsou silně bisimulární právě tehdy, když splňují stejnou množinu formulí v HML [1].

2.6 Rozšíření a další alternativy

Kalkulus komunikujících systémů (CCS) byl jednou z prvních procesních algeber a položil teoretické základy verifikace. S postupným rozvojem teoretické informatiky však vznikaly i další rozšíření nebo úplně nové matematické formalismy, které na CCS navazují.

2.6.1 CCS s předáváním hodnot

Základní verze CCS, jak byla představena v předchozích kapitolách, modeluje komunikaci pouze jako vzájemné odeslání signálu na určitém kanálu (tzv. handshake synchronizace), aniž by při této události docházelo k výměně jakýchkoli stavových dat. V reálném světě si však procesy mezi sebou potřebují vyměňovat konkrétní datové hodnoty, jako jsou například celá čísla, stavová data či textové řetězce. Pro tyto účely existuje rozšíření zvané CCS s předáváním hodnot (*Value-passing CCS*), ve kterém se komunikace obohacuje o parametry [3].

Přijetí hodnoty x na portu in se modeluje jako $in(x).P$, zatímco odeslání konkrétní hodnoty v (či výrazu) na portu out se zapisuje jako $\overline{out}(v).P$. Důležitým bodem je, že pokud v modelu pracujeme pouze s omezeným konečným počtem hodnot datového typu, nepřináší nám toto rozšíření z teoretického hlediska žádnou novou expresivitu. Každý takový proces lze totiž přeložit do standardního CCS tak, že se příjem hodnot nahradí velkou sadou nedeterministických výběrů pro každou možnou hodnotu datového typu a odeslání by se mapovalo na staticky definovaný specifický název akce (např. odeslání čísla 2 jako akce $\overline{out(2)}$).

2.6.2 Alternativy pro reaktivní systémy

Paralelně s vývojem CCS vznikala i procesní algebra CSP (*Communicating Sequential Processes*), jejímž autorem je sir C. A. R. Hoare [5]. Zatímco v CCS je komunikace založena primárně na komplementárních zprávách synchronizovaných přes neviditelnou akci τ, CSP vnímá komunikaci jako sdílení událostí, do kterých se může synchronně zapojit i více paralelních procesů současně (tzv. *multi-way synchronization*). Abstrakce, kterou algebra CSP zavedla, se v softwarovém inženýrství ujala a výrazně ovlivnila návrh moderních programovacích jazyků. Příkladem je jazyk Go, který se z CSP přímo inspiroval při návrhu svých `goroutines` a komunikačních kanálů.

Dalším významným pohledem na modelování komunikujících systémů je algebraický přístup, reprezentovaný formalismem ACP (*Algebra of Communicating Processes*), který vytvořili Jan Bergstra a Jan Willem Klop [6]. Ten, na rozdíl od CCS, které zakládá svou sémantiku zespodu (*bottom-up*) induktivně pomocí strukturální operační sémantiky (SOS) a důkazových stromů, popisuje chování systémů primárně axiomaticky shora (*top-down*). Chování systému a ekvivalence procesů jsou v ACP formálně odvozovány převážně dedukcí ze sady definovaných rovnic tvořících danou algebru.

Snad nejvýznamnějším pokročilým formalismem, zavedeným autorem CCS Robinem Milnerem na začátku devadesátých let, je π-kalkul (*Pi-calculus*) [7]. Tento kalkul byl vyvinut jako odpověď na nutnost modelovat takzvané mobilní procesy, tedy systémy, u kterých se může dynamicky za běhu měnit síťová a komunikační topologie. Zatímco v klasickém CCS jsou komunikační kanály (jména akcí) staticky svázané s architekturou systému, π-kalkul umožňuje předávat samotná jména kanálů jako komunikační data skrze jiné kanály (tzv. *link passing* či *name passing*). Tento mechanismus dynamického vytváření, delegování a sdílení nových referencí v čase z něj dělá vysoce expresivní nástroj

pro modelování moderních distribuovaných internetových architektur, mikroservis a bezdrátových sítí s proměnlivým připojením klientů.

Kapitola 3

Analýza požadavků

Tato kapitola se věnuje analýze požadavků na vyvíjenou aplikaci, jejímž cílem je podpora výuky vybraných problémů z oblasti formální verifikace systémů. V následujících sekcích je specifikován primární účel aplikace, zhodnocení existujících nástrojů a podrobná specifikace funkčních i nefunkčních požadavků.

3.1 Účel aplikace

Primárním účelem vyvíjené aplikace je sloužit jako interaktivní vizuální pomůcka pro studenty předmětu Modelování a verifikace (MaV), případně pro studenty připravovaného předmětu Formální Verifikace Systémů. Teorie formální verifikace a modelování konkurentních systémů pomocí kalkulu komunikujících systémů může být pro studenty zpočátku náročnější na pochopení, zejména pokud nemají k dispozici adekvátní nástroj pro vizualizaci chování těchto systémů.

Hlavním přínosem aplikace je provedení studenta prvními fázemi studia problematiky. Uživatel si může interaktivně vyzkoušet sestrojení ohodnoceného přechodového systému na základě definovaných CCS výrazů, případně krok za krokem dokazovat platnost přechodů za užití pravidel strukturální operační sémantiky.

Jedním z hlavních cílů aplikace je tento vzdělávací proces usnadnit poskytnutím okamžité vizuální zpětné vazby. Aplikace umožní studentům experimentovat s vlastními procesy, vizualizovat jejich syntaktickou strukturu, krokovat jejich běh a především interaktivně budovat formální důkazy pomocí SOS pravidel. Díky tomu si mohou uživatelé lépe představit abstraktní principy synchronizace, nedeterminismu a komunikace mezi procesy.

3.2 Analýza existujících řešení

Oblast formální verifikace a analýzy systémů popsaných pomocí CCS již disponuje řadou aplikací. Většina z nich však byla navržena primárně pro výzkumné nebo průmyslové účely, nikoliv explicitně

jako pomůcky pro začátečníky. Mezi nejvýznamnější programy v kontextu existujícího softwaru je vhodné zmínit následující aplikace.

3.2.1 Concurrency Workbench Aalborg (CAAL)

Jako nejvýznamnější moderní nástroj lze uvést CAAL (Concurrency Workbench Aalborg) [8]. Jedná se o komplexní webovou aplikaci umožňující specifikaci procesů v CCS, generování LTS, ověřování bisimulačních ekvivalencí a model checking pomocí modální logiky. Je to poměrně intuitivní nástroj, který se i během výuky předmětu používá, obsahuje však i určité limity. Hlavní nevýhodou je absence podpory pro dokazování přechodů pomocí jednotlivých SOS pravidel, což omezuje jeho využití v úvodních fázích výuky. Při generování LTS grafů navíc CAAL využívá algoritmy optimalizované spíše pro rozsáhlé stavové prostory, což má za následek horší přehlednost u menších výukových příkladů. Nástroj rovněž neposkytuje vizualizaci abstraktního syntaktického stromu (AST), která je pro pochopení sémantiky a priorit operátorů klíčová.

3.2.2 Edinburgh Concurrency Workbench (CWB)

Historicky důležitým nástrojem, který definoval standard pro automatizovanou analýzu CCS, je CWB [9]. Ačkoliv se jedná o spolehlivý software s vysokým výkonem, v dnešní době již není dále vyvíjen. Na tento nástroj později navázal projekt Concurrency Workbench of the New Century (CWB-NC), který přinesl další rošíření a vylepšení architektury, avšak i jeho vývoj byl nakonec opuštěn. Zásadním nedostatkem obou těchto nástrojů pro účely moderní výuky je zastaralé uživatelské rozhraní založené výhradně na příkazové řádce. Neposkytují žádnou formu interaktivní vizualizace grafů či syntaktických stromů, nejsou tedy pro výuku a vizualizaci příliš v vhodné.

3.2.3 Zhodnocení a opodstatnění nového řešení

Z analýzy existujících řešení vyplývá, že jejich hlavním nedostatkem je složitost uživatelského rozhraní a absence nástrojů zaměřených čistě na pochopení základních principů (např. krokové budování SOS důkazů a vizualizace AST). Hlavním opodstatněním vývoje nového řešení je tak primárně zaměření na výuku. Aplikace nabídne validaci syntaxe v reálném čase, možnost vizuální nápovědy při volbě SOS pravidel a moderní, čisté prostředí. Stane se tak vhodným doplňkem (zejména pro úvodní přednášky a cvičení) k robustnějším nástrojům, nikoliv však jejich plnohodnotnou náhradou.

3.3 Funkční požadavky

Funkční požadavky přímo definují chování a možnosti vyvíjené aplikace. Vycházejí z oficiálního zadání diplomové práce a jsou rozšířeny o potřeby nezbytné pro reálné nasazení ve výuce:

1. **Zadávání a syntaktická analýza CCS výrazů**

- Systém bude obsahovat textový editor pro zápis zdrojového kódu v syntaxi jazyka CCS.
- Aplikace zajistí v reálném čase syntaktickou analýzu vstupu a vizuálně upozorní uživatele na případné chyby (např. chybějící závorky, nepovolené znaky, nedefinované procesy).
- K zadanému a syntakticky korektnímu výrazu systém umožní vygenerování a zobrazení syntaktického stromu, reprezentujícího hierarchickou strukturu výrazu a prioritu operátorů.

2. **Interaktivní konstrukce odvozovacího stromu (SOS)**

   - Pro uživatelem zvolené procesy X a Y a akci α systém uživateli poskytne nástroje pro sestavení formálního důkazu přechodu $X \xrightarrow{\alpha} Y$ na základě axiomů a odvozovacích pravidel strukturální operační sémantiky.
   - Tento proces musí probíhat interaktivně: uživatel bude postupně vybírat pravidla aplikovatelná na daný podvýraz, včetně jejich příslušných parametrů.
   - Aplikace nabídne dva režimy podpory výuky:
     (a) *Režim bez nápovědy:* Uživatel provádí volbu pravidel samostatně. V případě chybného kroku jej aplikace pouze upozorní, že dané pravidlo s příslušnými parametry nelze použít.
     (b) *Režim s nápovědou:* Systém na základě aktuálního stavu výrazu interaktivně filtruje a zvýrazní pouze ta SOS pravidla, která jsou syntakticky aplikovatelná, a omezí výběr parametrů na validní možnosti.

3. **Generování a vizualizace ohodnoceného přechodového systému (LTS)**

   - Na základě zadaného CCS výrazu systém vygeneruje odpovídající stavový prostor a zobrazí jej jako orientovaný graf, kde vrcholy reprezentují stavy prostoru a orientované hrany jednotlivé akce.
   - Systém musí být odolný vůči zacyklení v případech, kdy je generovaný LTS nekonečný (např. vlivem neomezeného vytváření nových paralelních komponent nebo nezastavené rekurze). Generování bude omezeno vhodným limitem hloubky či počtu stavů tak, aby se předešlo zamrznutí uživatelského rozhraní.

4. **Simulace běhu procesů**

   - Aplikace musí obsahovat interaktivní simulátor umožňující manuální krokování běhu procesu napříč vygenerovaným LTS grafem.
   - Uživatel bude mít možnost v každém stavu zvolit, kterou z dostupných akcí chce provést.
   - Simulátor dovolí návrat o krok zpět, případně vrácení celé simulace do výchozího stavu.

- Při analýze paralelních procesů (využívajících operátor $|$) simulátor vizuálně demonstruje nezávislé provádění akcí i interní komunikaci (handshake) přes komplementární porty (např. α a $\overline{\alpha}$), vedoucí ke vzniku tiché akce τ.

5. **Předdefinované ukázkové vstupy**
   - Součástí systému bude sada předpřipravených příkladů demonstrujících použití jednotlivých operátorů CCS a typických situací ve formální verifikaci.
   - Tyto ukázky musí být možné načíst na jedno kliknutí bez nutnosti psaní kódu, což výrazně urychlí seznámení uživatele s aplikací.

3.4 Vedlejší požadavky

Vedlejší (nefunkční) požadavky definují vlastnosti systému, které nesouvisejí přímo s hlavní funkcionalitou verifikace, ale jsou klíčové pro celkovou uživatelskou přívětivost (UX), stabilitu a moderní standardy vývoje webových aplikací.

1. **Podpora offline režimu a PWA architektura:** Aplikace bude navržena jako progresivní webová aplikace (PWA). Logika výpočtů a zpracování CCS poběží primárně na straně klienta, což umožní téměř plnohodnotné využití nástroje i bez připojení k internetu. Ukládání rozpracovaných příkladů zajistí lokální úložiště prohlížeče, aby bylo možné uchovat rozpracovaný stav přímo v zařízení i bez nutnosti přístupu k internetu.

2. **Lokalizace uživatelského rozhraní (i18n):** Aplikace nabídne podporu lokalizace minimálně pro češtinu a angličtinu. Přepnutí jazyka proběhne plynule bez nutnosti opětovného načtení celé stránky. Architektura aplikace by měla umožnit snadné přidání dalších jazyků v budoucnosti.

3. **Export a vizualizace dat:** Aplikace umožní export vygenerovaných syntaktických stromů a LTS grafů. Podporován bude export do rastrových (PNG) i vektorových (SVG) formátů, aby je studenti mohli snadno začlenit do svých úkolů či prací. Dostupný bude i export struktury do strojově čitelného formátu (JSON).

4. **Sdílení programů:** Systém poskytne mechanismus pro snadné sdílení vytvořených procesů. Uživatel získá možnost jednoduše zkopírovat kód nebo sdílet rozpracované řešení, což zefektivní komunikaci mezi uživateli.

5. **Přístupnost a responsivní design:** Přestože se předpokládá primární využití na desktopových zařízeních, rozhraní bude responsivní a funkční i na mobilních telefonech a tabletech. Aplikace by měla být také přístupná v souladu se zákonem o přístupnosti, platným od roku 2025. I přesto, že se na tuto aplikaci zákon momentálně nevztahuje, měla aspoň částečně respektovat metodiku WCAG 2.2 [10].

6. **Integrovaná nápověda a dokumentace:** Součástí uživatelského rozhraní bude kontextová nápověda (např. tooltipy vysvětlující jednotlivá SOS pravidla) a centrální dokumentace, která stručně objasní jak ovládání samotné aplikace, tak i základní pojmy.

7. **Moderní UI/UX s využitím animací:** Vizuálně by stránka měla působit čistě a moderně. Přechody stavů a úpravy grafů budou podpořeny plynulými animacemi. Tyto animace nebudou plnit pouze estetickou funkci, ale poslouží především ke snížení kognitivní zátěže uživatele (např. plynulé zobrazení kroků při animaci průchodu grafem).

8. **Možnost uzamknutí programu:** Rozhraní umožní dočasné uzamčení textového editoru, zadání SOS důkazů a dalších prvků aplikace. Toto uzamčení bude sloužit zejména jen jako ochrana proti nechtěným změnám, uživatel si jej bude moci u programů libovolně aktivovat či deaktivovat.

9. **Strukturální redukce LTS grafu:** Systém uživateli nabídne možnost zjednodušit generované grafy a stromy, například automatickým odstraňováním zbytečných `Nil` procesů či aplikací základních pravidel strukturální kongruence.

10. **Automatické dokázání SOS přechodů:** Uživatel si bude moct nechat při důkazu přechodu za pomoci SOS pravidel v režimu s nápovědou automaticky vygenerovat kompletní důkaz přechodu, pokud je tedy tento přechod vůbec platný.

Kapitola 4

Použité technologie

V této kapitole jsou popsány softwarové nástroje, knihovny a technologie, které byly využity při návrhu a implementaci praktické části této práce. Vzhledem k požadavkům na interaktivitu a vizualizaci formálních modelů byly zvoleny moderní webové technologie, které zajistí vysokou modularitu a přívětivé uživatelské rozhraní.

4.1 Vývojové prostředí a sestavení aplikace

4.1.1 Visual Studio Code

Jako primární vývojové prostředí byl zvolen editor Visual Studio Code (VS Code) od společnosti Microsoft [**TODO**]. Jedná se o open-source editor, který se stal standardem pro vývoj webových aplikací. Jeho hlavní výhodou v kontextu této práce je nativní podpora jazyka TypeScript a Node.js. Editor nabízí zabudovanou spolupráci s verzovacím nástrojem Git a integrovaný terminál, přes který je pak možné spouštět lokální vývojový server nebo skripty pro sestavení aplikace, bez nutnosti opouštět okno editoru.

4.1.2 GitHub

Pro verzování zdrojových kódů a zálohování projektu byl využit systém Git v kombinaci s platformou GitHub [**TODO**]. Využití repozitáře na této platformě zajistilo nejen transparentní historii vývoje pro vedoucího práce, ale také bezpečné a snadno dostupné úložiště pro zdrojové kódy.

Platforma GitHub nabízí také spoustu rozšíření, příkladem může být služba GitHub Pages [**TODO**], která byla v této práci využita pro kontinuální nasazení (Continuous Deployment) webové aplikace. Pro usnadnění přístupu k aplikaci byla k repozitáři připojena vlastní doména. Aktuální verze aplikace je tedy dostupná na adrese `ccs.koberskyj.cz`. Využití služby GitHub Pages poskytuje stabilní a ověřené hostingové řešení, které garantuje vysokou dostupnost aplikace pro uživatele.

4.1.3 Node.js a npm

Node.js je běhové prostředí (runtime environment) postavené na enginu V8 z prohlížeče Google Chrome, které umožňuje spouštět JavaScript kód mimo webový prohlížeč [**TODO**]. V tomto projektu neslouží Node.js pro běh serverové části (backendu), jelikož je aplikace navržena jako čistě klientská, ale poskytuje infrastrukturu pro běh vývojových a sestavovacích nástrojů.

Společně s Node.js byl využit balíčkovací systém `npm` (Node Package Manager), který spravuje všechny externí závislosti projektu. Pomocí `npm` byly do projektu integrovány veškeré knihovny zmíněné v této kapitole. Správa verzí v souboru `package.json` navíc zajišťuje deterministická sestavení aplikace na různých počítačích s různým operačním systémem.

4.1.4 Vite

Pro samotné sestavení a běh vývojového serveru byl použit nástroj Vite [**TODO**]. Oproti starším bundlerům, jako je například Webpack, přináší Vite značné zrychlení vývojového procesu. Využívá nativních ES modulů v prohlížeči, což znamená, že při úpravě kódu není nutné překompilovávat celou aplikaci.

Změny provedené ve zdrojovém kódu se díky funkci Hot Module Replacement (HMR) projevují v prohlížeči téměř okamžitě. Ve fázi produkčního sestavení pak Vite využívá nástroj Rollup, který kód optimalizuje, minifikuje a připraví pro statické nasazení, například právě na službu GitHub Pages.

4.2 Základní webové technologie a uživatelské rozhraní

Klientská část aplikace musí být, podle požadavků specifikovaných v předchozí kapitole, dynamická a schopná okamžitě reagovat na vstupy uživatele. To je klíčové například při interaktivní konstrukci SOS důkazů, kdy je nezbytné okamžitě validovat aplikovatelnost sémantických pravidel. Z tohoto důvodu byla zvolena architektura Single Page Application (SPA). V tomto modelu se veškeré HTML, JavaScript a CSS soubory načtou pouze jednou a obsah stránky je pak dynamicky přepisován na základě interakce uživatele bez nutnosti znovunačtení celé webové stránky ze serveru.

4.2.1 React.js a TypeScript

Pro vývoj uživatelského rozhraní byla použita JavaScript knihovna React.js, vyvinutá společností Meta [**TODO**]. React umožňuje tvorbu komplexních uživatelských rozhraní skládáním malých, znovupoužitelných a izolovaných komponent. Deklarativní přístup Reactu výrazně usnadňuje synchronizaci vnitřního stavu aplikace (například informace o aktuálním kroku v SOS důkazu nebo vizuálním stavu LTS grafu) s tím, co uživatel vidí na obrazovce.

Knihovna React byla také doplněna o jazyk TypeScript, který obohacuje standardní JavaScript o statické typování.

Při návrhu React komponent byly navíc uplatňovány principy metodiky SOLID, což je sada doporučení pro psaní znovupoužitelných komponent [**TODO**]. Jedná se například o jasné vymezení odpovědnosti komponenty nebo o to, aby byly komponenty otevřené pro vnější rozšíření. Příkladem takovéto komponenty může být `CCSViewer`, který umožňuje zvýraznit syntaxi vloženého CCS výrazu a zobrazit jej jako klasický textový element, jakým je například značka `span`.

4.2.2 Tailwind CSS

Pro stylování aplikace byl využit framework Tailwind CSS [**TODO**]. Na rozdíl od tradičních CSS frameworků (jako je např. Bootstrap), které poskytují předpřipravené komponenty, je Tailwind „utility-first“ framework. Poskytuje nízkoúrovňové třídy (např. pro nastavení okrajů, barev či flexboxu), které se aplikují přímo do HTML/JSX kódu komponent.

Tento přístup eliminuje nutnost neustálého přepínání mezi oddělenými soubory se styly a aplikační logikou. Vývojář si tak může při pohledu na zdrojový kód komponenty rovnou představit její rozvržení. Zásadní výhodou knihovny Tailwind je přítomnost „Just-In-Time“ kompilátoru, který při sestavování aplikace prohledá kód a do výsledného generovaného CSS souboru zahrne výhradně ty styly, které byly skutečně použity. Tím je zajištěna minimální výsledná velikost aplikace.

4.2.3 Shadcn

Zatímco Tailwind poskytuje nástroje pro základní formátování, pro komplexnější interaktivní prvky uživatelského rozhraní byla využita kolekce komponent Shadcn/ui [**TODO**]. Nejedná se o tradiční knihovnu, která se instaluje přes `npm`, ale spíše o sadu přizpůsobitelných komponent generovaných přímo do zdrojového kódu projektu, nad kterými má vývojář plnou kontrolu.

Komponenty z rodiny Shadcn jsou postavené na knihovně Radix UI [**TODO**], což garantuje jejich vysokou přístupnost (accessibility) pro uživatele s omezením, včetně podpory pro ovládání z klávesnice a čtečky obrazovky. V aplikaci byly tyto prvky využity pro tvorbu rozbalovacích nabídek, tlačítek, modálních oken a vyskakovacích upozornění.

4.3 Zpracování a vizualizace formálních modelů

Hlavní cíl této práce spočívá ve schopnosti aplikace syntakticky analyzovat uživatelem zadané CCS procesy a vizualizovat odpovídající datové struktury. K tomuto účelu byly využity specifické knihovny pro zpracování textu a interaktivní vykreslení grafů.

4.3.1 Peggy.js

Pro zpracování zadaných programů v kalkulu komunikujících systémů byl využit generátor syntaktických analyzátorů Peggy.js [**TODO**]. Tento nástroj umožňuje definovat formální gramatiku jazyka

pomocí zápisu PEG (Parsing Expression Grammar). Z této deklarativní definice Peggy.js následně vygeneruje deterministický a vysoce efektivní parser přímo v jazyce JavaScript.

V aplikaci je tento parser zodpovědný za prvotní kontrolu syntaktické správnosti zadaných CCS výrazů (validace závorek, prefixů, operátorů apod.). V případě chybného vstupu dokáže vygenerovat detailní chybová hlášení identifikující řádek a sloupec chyby, čímž uživateli poskytuje okamžitou zpětnou vazbu srovnatelnou s moderními vývojovými prostředími.

Při úspěšném zpracování transformuje Peggy.js vstupní text do struktury abstraktního syntaktického stromu. Tento strom je následně zkontrolován dodatečnými sémantickými kontrolami (např. kontrola existence deklarací všech volaných procesních konstant). Po úspěšné validaci tvoří AST základní datovou strukturu pro generování LTS grafu a pro konstruování jednotlivých kroků strukturální operační sémantiky.

4.3.2 CodeMirror

O zadávání a zobrazování zdrojových kódů CCS programů se stará editorová komponenta CodeMirror [**TODO**]. Jedná se o flexibilní webový textový editor navržený pro implementaci vývojářských nástrojů přímo v prohlížeči.

Pro potřeby této práce byl vytvořen vlastní režim lexikální analýzy (syntax highlighting), specifický pro jazyk CCS, jehož pravidla odpovídají gramatice použité v Peggy.js. Díky tomu jsou vizuálně odděleny akce, operátory, závorky, názvy procesů a speciální tichá akce τ. Kromě hlavního okna pro zápis kódu je CodeMirror v aplikaci využit i v režimu pouze pro čtení (read-only) k formátování kratších úseků textu. To umožňuje zachovat konzistentní zvýraznění syntaxe v celém uživatelském rozhraní, od instrukcí a nápověd, až po samotný text interaktivních SOS důkazů.

4.3.3 Cytoscape.js a doplňky

Pro vizualizaci ohodnocených přechodových systémů a syntaktických stromů, z nichž mohou vzniknout rozsáhlé orientované grafy, byla použita knihovna Cytoscape.js [**TODO**]. Jedná se o vysoce optimalizovanou JavaScript knihovnu určenou primárně pro modelování a interaktivní vizualizaci komplexních síťových struktur.

V rámci aplikace Cytoscape vykresluje stavy reprezentované jako vrcholy grafu a přechody definované akcemi ve formě orientovaných hran. Knihovna poskytuje uživateli možnost graf volně přibližovat, oddalovat a posouvat. Dále umožňuje pokročilou interaktivitu, jako je dynamické propojení syntaktického stromu s textovým editorem CodeMirror, kdy se při najetí myší na vrchol grafu zvýrazná odpovídající část kódu.

Pro rozšíření možností vizualizace byly využity následující doplňky (pluginy) knihovny:

- **cytoscape-dagre:** Rozšíření, které integruje algoritmus Dagre pro hierarchické a deterministické rozvržení grafu. Tento algoritmus je využit při vizualizaci postupně rozvíjených stavových prostorů LTS. Detailněji je tento algoritmus popsán v kapitole 6.1.4.

- **cytoscape-svg (cysvg):** Tento doplněk doplňuje aplikaci o funkcionalitu exportu vykresleného grafu do bezztrátového vektorového formátu SVG.

4.4 Správa dat

Jelikož je aplikace z architektonického hlediska navržena bez serveru (serverless klient), neobsahuje tradiční backendovou relační databázi. Veškerá data, nastavení i modely jsou ukládány a spravovány výhradně na straně klientského prohlížeče.

4.4.1 Formát JSON

Základním formátem pro strukturu a přenos všech dat v aplikaci je formát JSON (JavaScript Object Notation). Tento datový formát je nativně podporován jazykem JavaScript a je jednoduše čitelný jak pro stroj, tak pro člověka. V architektuře projektu je použit na několika místech:

- Slouží jako základní formát pro ukládání všech vytvořených CCS programů, což umožňuje jejich snadné zálohování či sdílení mezi uživateli.
- Využívá se pro uložení předpřipravených vstupů (soubory `baseProgramList.json` a `supplementaryPrograms.json`), kde je definována sada validních CCS programů demonstrujících různé příklady z oblasti paralelních systémů.
- Poskytuje strukturu pro správu lokalizace v rámci uživatelského rozhraní.
- Nabízí dodatečnou možnost strojového exportu vygenerovaných grafů z knihovny Cytoscape, díky čemuž lze s těmito daty dále pracovat v externích analytických nástrojích.

4.4.2 LocalStorage

Pro persistenci dat mezi jednotlivými relacemi uživatele (např. po zavření nebo obnovení stránky) je využíváno webové rozhraní `localStorage`, které je součástí Web Storage API. Jedná se o lokální „key-value“ úložiště alokované prohlížečem a izolované čistě pro danou webovou doménu.

Prostřednictvím `localStorage` aplikace uchovává veškerá uživatelská data, včetně seznamu vytvořených CCS programů, aktuálního nastavení lokalizace nebo preferencí zobrazení.

4.5 Doplňkové nástroje

4.5.1 Lokalizace

Jelikož se problematika formální verifikace a kalkulu komunikujících systémů může vyučovat v českém i anglickém jazyce, byla aplikace od začátku navržena s podporou pro internacionalizaci (i18n).

Textové řetězce nejsou pevně zakomponovány v logice komponent, ale jsou externě a dynamicky načítány pomocí lokalizačního frameworku `i18next` [**TODO**]. Tato architektura zajišťuje spolehlivý překlad veškerých prvků uživatelského rozhraní, chybových zpráv i nápověd do češtiny a angličtiny, a to se schopností okamžitého přepnutí jazyka za běhu aplikace, bez nutnosti obnovení stránky.

4.5.2 Progresivní webová aplikace

S cílem maximalizovat dostupnost nástroje byla celá aplikace navržena jako progresivní webová aplikace (PWA). Tohoto konceptu bylo dosaženo integrací modulu `vite-plugin-pwa`.

Architektura PWA využívá Service Workerů, což jsou skripty běžící odděleně na pozadí prohlížeče, které se starají o proaktivní ukládání (caching) statických souborů, knihoven i lokalizačních dat do mezipaměti. Aplikace tak po prvním načtení může plnohodnotně fungovat i bez připojení k internetu, a to včetně kompilace výrazů, tvorby grafů a ukládání programů. Součástí konfigurace je i webový manifest, který uživatelům dává možnost „nainstalovat“ si aplikaci přímo do jakéhokoli operačního systému svého zařízení. Aplikace se následně spouští ve vlastním vyhrazeném okně bez prvků internetového prohlížeče.

4.5.3 Lighthouse

Při vývoji aplikace byl pravidelně využíván analytický nástroj Google Lighthouse [**TODO**]. Jedná se o automatizovaný open-source nástroj určený pro auditování a vylepšování kvality webových stránek.

Lighthouse simuluje průchod aplikací a generuje detailní metriky pokrývající výkon, SEO a dodržení standardů webových stránek. V rámci této práce byl nástroj primárně využit také k identifikaci slabých míst v oblasti přístupnosti (accessibility). Na základě vygenerovaných reportů byly v aplikaci provedeny úpravy týkající se barevného kontrastu, sémantiky HTML značek a dostupnosti formulářových prvků pro čtečky obrazovky. Cílem bylo maximalizovat skóre v těchto metrikách a zajistit, aby byla aplikace připravená na případné produkční nasazení, kde by již musela splňovat zákon o přístupnosti, zmíněný v kapitole 3.3.

Kapitola 5

Návrh řešení

Tato kapitola se věnuje návrhu webové aplikace, která přímo vychází z teoretických poznatků o reaktivních systémech a z analýzy požadavků definovaných v kapitole 3. Cílem tohoto návrhu je vytvořit modulární a uživatelsky přívětivé prostředí, které studentům i vyučujícím usnadní práci s formálními modely. Architektura je koncipována tak, aby oddělovala prezentační logiku od výpočetního jádra, což umožňuje flexibilní rozšiřování aplikace. Návrh konkrétní implementace a detailní popis použitých algoritmů jsou následně předmětem kapitoly 6.

5.1 Případy užití

Pro správné navržení uživatelského rozhraní a aplikační logiky je nezbytné definovat, jakým způsobem budou uživatelé s aplikací interagovat. Systém je navržen tak, aby spolehlivě obsloužil několik základních scénářů pokrývajících potřeby studentů i pedagogů.

Mezi hlavní případy užití ze strany uživatele jako studenta patří:

- **Samostatné procvičování látky:** Student si může ověřit a upevnit své znalosti z oblasti formální verifikace. Po spuštění aplikace má k dispozici sadu předpřipravených příkladů, dostupné na jedno kliknutí. Na těchto příkladech má možnost krokovat odvozování syntaxe, vizualizovat grafy nebo interaktivně dokazovat přechody pomocí pravidel strukturální operační sémantiky.
- **Řešení zadaných úkolů:** Student obdrží od vyučujícího konkrétní příklady (například ve formě exportovaného `.json` souboru), které snadno naimportuje do svého pracovního prostředí. V něm danou problematiku analyzuje, příklad vyřeší a následným exportem odevzdá vyučujícímu.
- **Tvorba vlastních příkladů:** Uživatel může v aplikaci založit zcela nový projekt, ve kterém si nadefinuje vlastní procesy kalkulu komunikujících systémů, odpovídající LTS grafy či důkazy přechodů. Tato funkcionalita je využitelná například při řešení zápočtových projektů nebo pokud si chce student sám něco zkusit vytvořit k procvičení.

Případy užití z pohledu vyučujícího zahrnují:

- **Plný rozsah studentských případů užití.**
- **Interaktivní ukázky ve výuce:** Vyučující může aplikaci aktivně prezentovat během přednášek či cvičení. Namísto vypisování důkazů na tabuli lze v aplikaci demonstrovat přechod mezi stavy krok za krokem, striktně podle formálních pravidel SOS. Tento přístup může výuku zrychlit a studenty motivovat, aby si předváděné postupy následně zkusili dokázat sami.
- **Příprava výukových materiálů:** Pedagog může v aplikaci navrhnout specifické modely LTS, které vizualizují nejrůznější stavy a přechody. Tyto modely lze následně exportovat ve formě obrázků nebo datových souborů, které se mohou dále využít například při tvorbě prezentací. Stejným způsobem byly vytvořeny obrázky 2.1 a 2.2 z kapitoly 2.

Po spuštění aplikace má uživatel rychlý přístup k nápovědě umístěné v pravé části hlavičky stránky, která objasňuje základní principy ovládání a fungování aplikace. Tuto nápovědu i celý program si také může přeložit tlačítkem volby jazyka, která se nachází vedle nápovědy. V hlavním rozcestníku si uživatel volí, zda chce pracovat s předdefinovaným programem, importovat existující, nebo vytvořit program zcela nový. Všechny tyto případy užití jsou ve zjednodušené podobě znázorněny na obrázku 5.1.

5.2 Návrh datových struktur

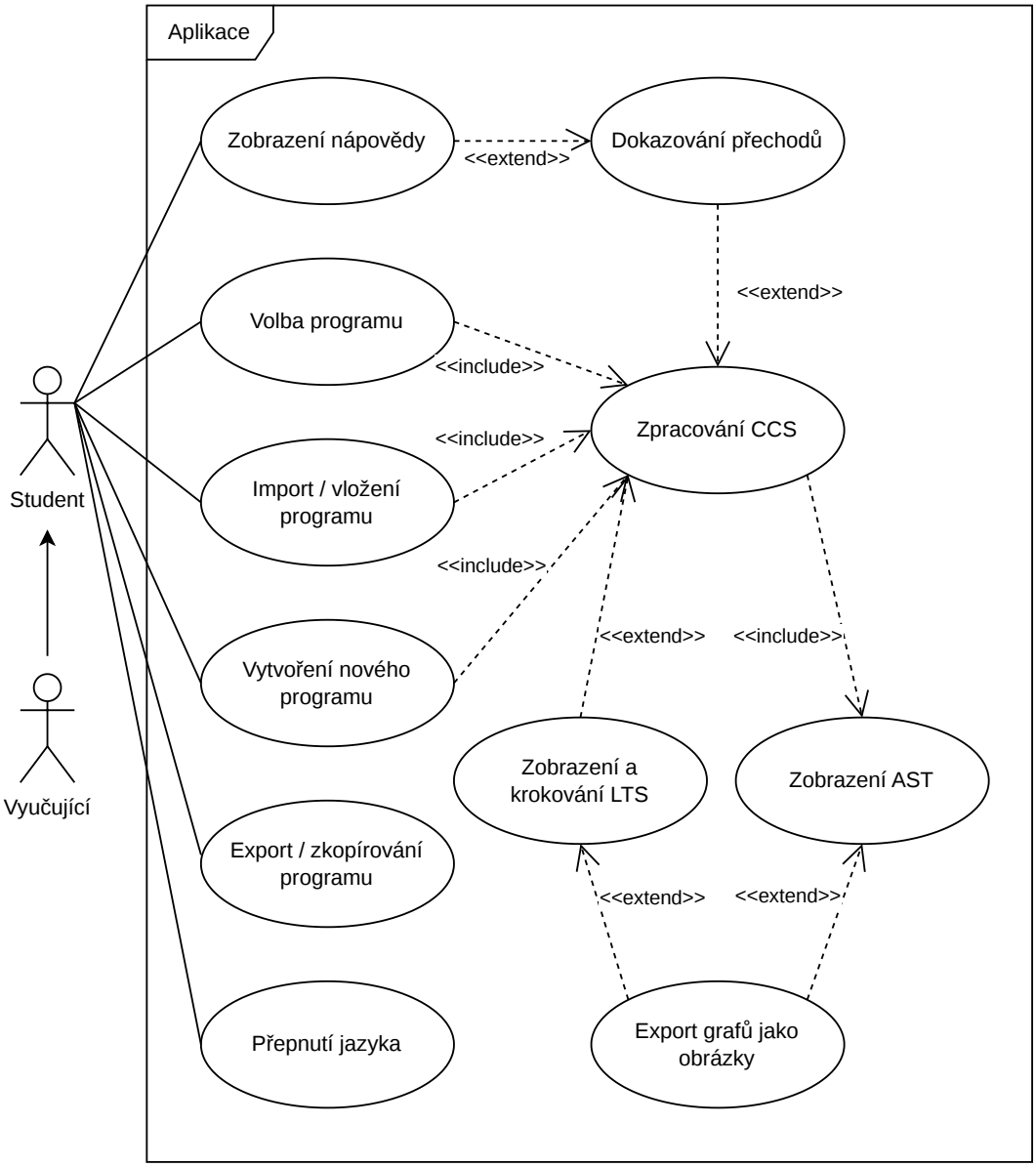
Pro efektivní abstrakci jednotlivých komponent aplikace bylo nutné navrhnout datové toky a vnitřní reprezentaci uživatelských vstupů. Z uživatelského hlediska aplikace přijímá jako vstup textovou definici CCS procesů. Tento vstup je v první fázi zpracován syntaktickým analyzátorem, který vstupní text transformuje do abstraktního syntaktického stromu ve formátu JSON.

Nad tímto stromem jsou posléze vystavěny další datové struktury nezbytné pro vnitřní logiku aplikace. Jedná se především o struktury pro generování vrcholů a hran grafu LTS, syntaktického stromu a struktury pro dokazování přechodů pomocí SOS pravidel. Detailní popis těchto struktur a procesu parsování je podrobněji rozebrán v kapitole 6.2.2.

5.3 Architektura webové aplikace

Aplikace je navržena jako Single-Page Application (SPA) s využitím knihovny React.js. Celý návrh dodržuje princip oddělení zodpovědností (Separation of Concerns). Zdrojové kódy a aplikační logika jsou logicky rozděleny do dvou hlavních vrstev, reprezentovaných adresářovou strukturou projektu:

- **Vrstva aplikační logiky (adresář `lib`):** Zajišťuje zapouzdření doménové logiky celého systému. Obsahuje pouze funkce, které nejsou závislé na uživatelském rozhraní. Nachází se



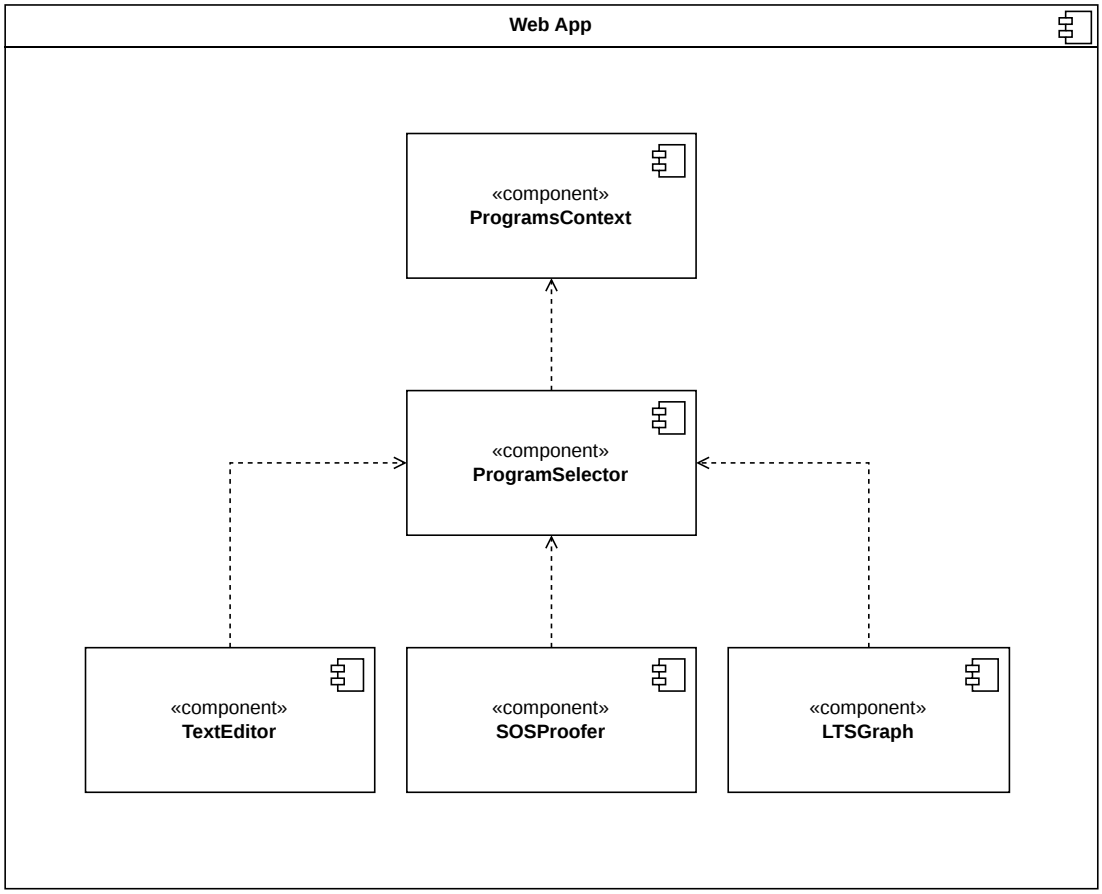
Obrázek 5.1: UML diagram hlavních případů užití aplikace

zde například zpracování uživatelského vstupu, generátor syntaktického stromu či algoritmy pro výpočet stavových prostorů i přechodů pro LTS, včetně přípravy struktury pro následné vykreslení grafu.

- **Vrstva uživatelského rozhraní (adresář `components`):** Vrstva sdružuje veškeré vizuální a interaktivní prvky aplikace, navržené jako znovupoužitelné komponenty. Součástí této vrstvy

jsou také UI prvky z knihovny `shadcn/ui`, které tak mohou být funkčně přizpůsobeny potřebám projektu.

Základní adresář komponent je dále hierarchicky členěn podle své funkčnosti, viz obrázek 5.2. Mezi základní funkční celky patří `TextEditor`, obalující knihovnu CodeMirror pro zápis CCS a syntakticky strom, `SOSProofer` pro vše potřebné z oblasti interaktivní konstrukce důkazů a `LTSGraph` pro všechny komponenty specifické pro zobrazení LTS grafu. Klíčovou roli hraje `ProgramsContext`, který centralizovaně spravuje stavy aplikace jakožto jediný zdroj pravdy (Single Source of Truth). Tímto přístupem se zamezuje desynchronizaci dat napříč komponentami. Veškeré úkony spojené s globální správou programů, jako je jejich vytváření, aktualizace, mazání a obecně práce s dynamicky generovanými kartami, jsou zpracovávany komponentami z `ProgramsSelector`.



Obrázek 5.2: UML diagram architektury aplikace

5.4 Uživatelské rozhraní

Návrh uživatelského rozhraní klade důraz na interaktivitu, plynulost a přehlednost. Uživatelské prostředí se skládá z jediné dynamické stránky, jejíž obsah je spravován prostřednictvím překryvných modálních oken a variabilního systému karet, zobrazených na obrázku 5.3.

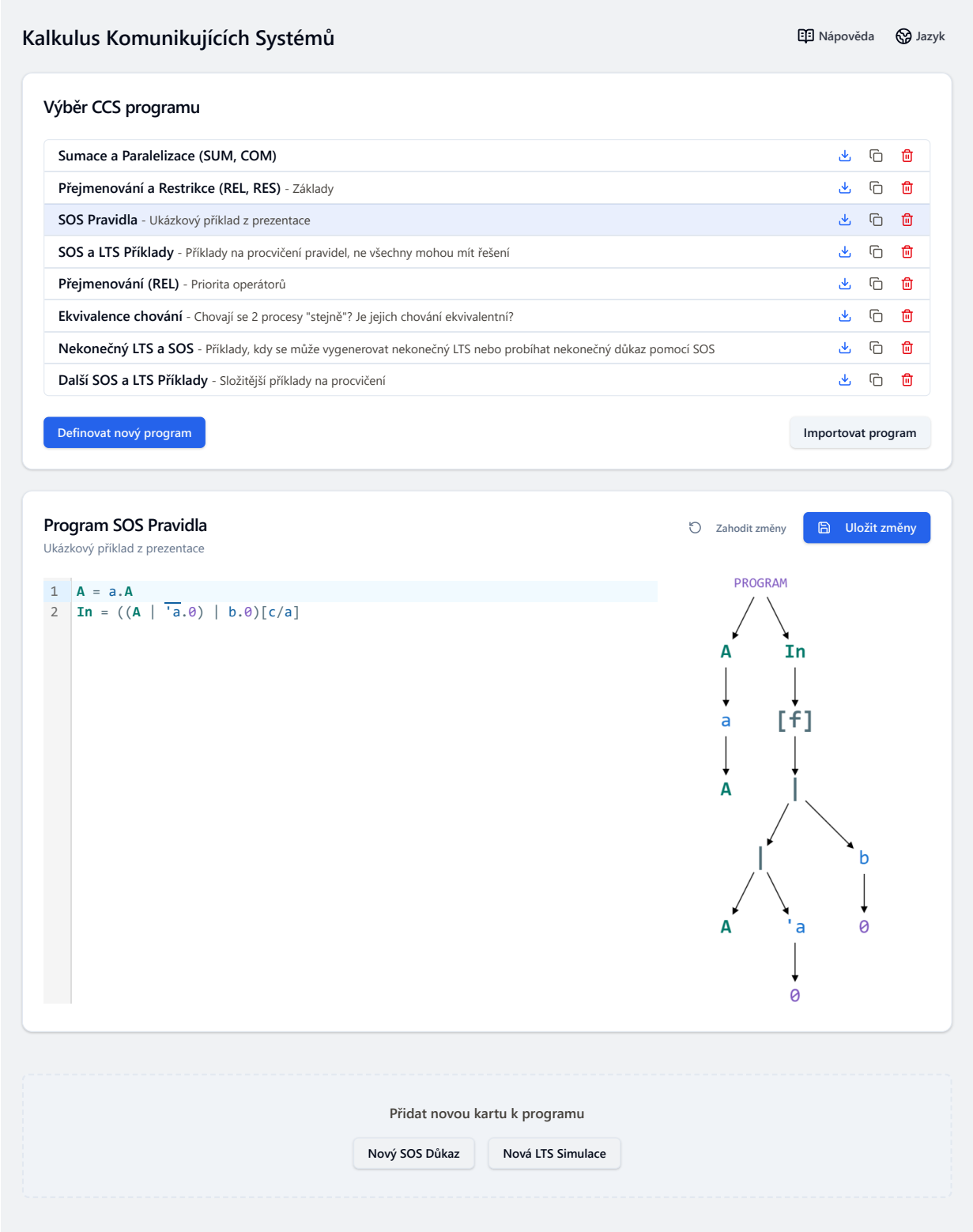
Při úvodním načtení aplikace vidí uživatel pouze výchozí kartu určenou k výběru programu. Zde má uživatel možnost načíst již vytvořený program, importovat vlastní zadání, nebo přes dialogové okno vytvořit program zcela nový. Hlavička stránky zůstává neměnná a nabízí změnu lokalizace i vyvolání nápovědy.

Jakmile je program zvolen či vytvořen, systém dynamicky vygeneruje hierarchicky podřazené karty. Výchozím a trvale přítomným prvkem je karta textového editoru. Uživatel v ní zadává definice procesů CCS, přičemž editor v reálném čase provádí validaci vstupu a graficky upozorňuje na případné syntaktické chyby. Při úspěšné validaci výrazu se ve stejné kartě rovněž automaticky vykresluje struktura syntaktického stromu. Kromě textového editoru může k jednomu programu náležet libovolné množství dalších podkaret. Ty mohou být dvou typů:

1. **Karta důkazu SOS:** Nabízí interaktivní prostředí pro krokovou konstrukci důkazu přechodu z jednoho CCS výrazu do druhého za využití formálních pravidel strukturální operační sémantiky.

2. **Karta LTS:** Zajišťuje vizualizaci ohodnocených přechodových systémů, kde jednotlivé vrcholy grafu reprezentují dosažitelné stavy programu a hrany odpovídají akcím, které mění jeho stav.

Na konci zobrazení podkaret je umístěna silueta karty, fungující jako rozcestník pro přidávání dalších karet obou zmíněných typů. Aby si uživatel udržel orientaci při vysokém počtu karet, umožňuje rozhraní každou z nich individuálně pojmenovat (funkcí editace v pravém horním rohu karty) a popsat tak, jakou konkrétní část systému daná karta vizualizuje či dokazuje. U vybraných karet je také dostupná nápověda pro specifické funkce karty. V případě, že je program explicitně uzamčen proti úpravám, přejde uživatelské rozhraní do režimu pouze pro čtení, kde není možné měnit obsah editoru a nelze jakkoli manipulovat s kartami.



Obrázek 5.3: Uživatelské rozhraní založené na systému karet

Kapitola 6

Implementace

6.1 Uživatelské rozhraní a logika aplikace

6.1.1 Výběr a správa programů

6.1.2 Karta textového editoru

6.1.3 Karta důkazu přechodů pomocí SOS pravidel

6.1.4 Karta zobrazení a krokování LTS grafu

6.2 Komunikace mezi komponentami

6.2.1 Programs store

6.2.2 Formát zpracovaného CCS výrazu

6.3 Možné úpravy aplikace

6.3.1 Přidání nových definic předdefinovaných programů

6.3.2 Lokalizace

6.4 Kompilace a nasazení

6.5 Optimalizace

6.5.1 Rychlost, Přístupnost, SEO

6.5.2 Vite komprimace

6.6 Možná vylepšení

Kapitola 7

Závěr

Literatura

1. ACETO, Luca; INGOLFSDOTTIR, Anna; LARSEN, Kim G; SRBA, Jiří. *Reactive systems: Modelling, specification and verification*. Cambridge University Press, 2007.
2. BAIER, Christel; KATOEN, Joost-Pieter. *Principles of model checking*. MIT press, 2008.
3. KOT, Martin. *Modelování a verifikace: Přednášky 1 až 5*. 2023.
4. MILNER, Robin. *Communication and concurrency*. Prentice-Hall, Inc., 1989.
5. GRAHAM, S; RIVEST EDITORS, R; HOARE, C. *Programming Teclmiques Communicating Sequential Processes* [online]. 1978. [cit. 2020-05-27]. Dostupné z: `https://www.cs.cmu.edu/~crary/819-f09/Hoare78.pdf`.
6. BERGSTRA, J.A.; KLOP, J.W. Algebra of communicating processes with abstraction. *Theoretical Computer Science* [online]. 1985, roč. 37 [cit. 2026-03-15]. Dostupné z DOI: `10.1016/0304-3975(85)90088-x`.
7. KOBAYASHI, Naoki; PIERCE, Benjamin C; TURNER, David N. Linearity and the pi-calculus. *ACM transactions on programming languages and systems* [online]. 1999-09, roč. 21, s. 914–947 [cit. 2024-06-02]. Dostupné z DOI: `10.1145/330249.330251`.
8. *CAAL* [online]. Cs.aau.dk, [b.r.]. [cit. 2026-03-15]. Dostupné z: `https://caal.cs.aau.dk/`.
9. *The Edinburgh Concurrency Workbench* [online]. Ed.ac.uk, [b.r.]. [cit. 2026-03-15]. Dostupné z: `https://homepages.inf.ed.ac.uk/perdita/cwb/`.
10. *10. Zákon o přístupnosti [Přístupné webové stránky]* [online]. Ipk.nkp.cz, 2019. [cit. 2026-03-15]. Dostupné z: `https://prirucky.ipk.nkp.cz/pristupnost/zakon_o_pristupnosti`.

Příloha A

Struktura přílohy